

Engineering Dynamic Scheduler for Work Manager

D Lesaint, N Azarmi, R Laithwaite and P Walker

Provision of service to business and residential customers, network maintenance and fault repair are core activities of large telecommunications companies which involve thousands of technicians every day. Managing a large-sized workforce efficiently in a highly dynamic environment is a challenge that BT has addressed by automating its work management processes within a single application known as Work Manager. For the most part, the success of Work Manager rests upon the ability of its work allocation component in determining the best assignments at any time and in any environment.

This paper presents a scheduler for Work Manager which is both predictive and reactive and whose computational model reconciles genericity and efficiency. Coupling Constraint Satisfaction and Local Search, this scheduler can generate long-term schedules that satisfy the conflicting objectives of feasibility, productivity, quality of service and cost minimisation. It also provides the capability to adjust schedules before despatch through on-line heuristic repair/improvement. Equipped with a powerful interface, this system has been successfully trialled within the field and is now deployed nationally.

1. Introduction

Major telecommunications companies employ a field workforce to serve customers, to maintain networks and to repair faults. This raises the issue of workforce management which can be summarised by the marching order ‘get the right technician, in the right place, at the right time, with the right stores, anytime, anywhere’. Workforce management is a major process for BT — 25 000 technicians perform 150 000 tasks every day across the UK — and the ability to deliver a high quality of service and of productivity, together with low operational costs, is vital to the success and competitiveness of the company as a whole.

For this reason, BT has automated its workforce management operations within a single application known as Work Manager. Among other operations, Work Manager automates the work allocation (deciding which task a technician should perform and when), the work despatch (informing a technician of his next assignment), and the work monitoring (feeding back information on the progress of tasks to the system).

This paper is concerned with the design of the work allocation capability. The previous capability was a real-time algorithm (RTA) that computed the next assignment for each technician in a reactive way [1]. Originally, RTA had been designed to schedule tasks with short response time commitments and identical skill requirements. However, business needs have changed and it has become necessary to handle a greater variety of tasks (provision and repair), skills and geography (customer premises, exchanges and local

network) and larger volumes of work (both reactive and proactive) so as to make Work Manager fully operational in different parts of BT’s business — residential customers, business customers, core network, etc (see Fig 1).

The reactive scheduling strategy and the short-term view implemented with RTA have now become inadequate to address all aspects of the workforce scheduling problem. The lack of forward view prevents the full-scale optimisation of global objectives such as the prioritisation of tasks within individual schedules, the long-term minimisation of ineffective time or the appropriate distribution of the workload across technicians and over time. Besides, scheduling one task ahead for each technician does not enable a proper synchronisation of inter-dependent tasks (e.g. sequential tasks). Finally, the absence of prediction in the schedules may be the source of excessive manual scheduling which contradicts the primary ambition of a fully automated work allocation process.

To make up for these limitations, a new solution has been developed — called Dynamic Scheduler — which is meant to be predictive as well as reactive. Dynamic Scheduler provides Work Manager with the capability to:

- construct, for a mobile and multi-skilled workforce, long-term work schedules that account for all the allocation constraints, quality objectives and business rules operated in Work Manager domains,

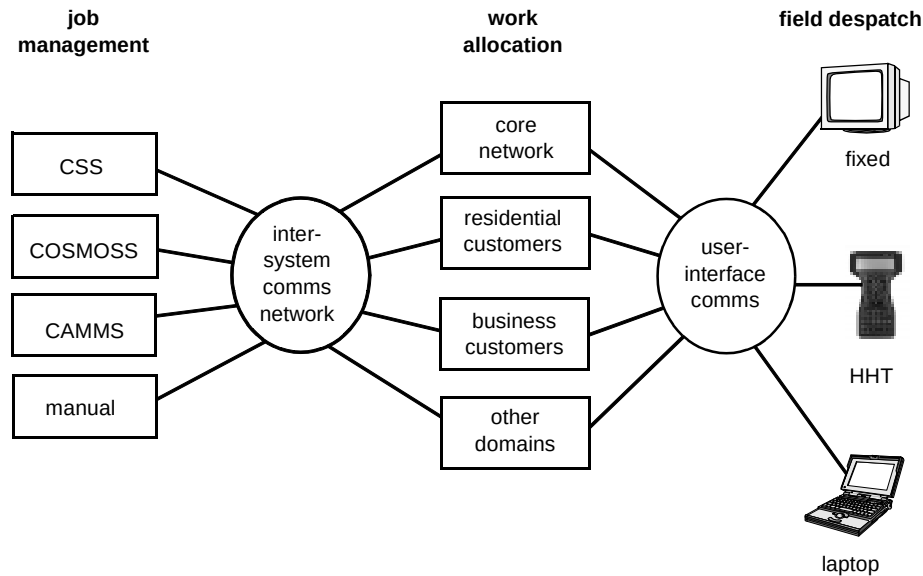


Fig 1 Work Manager.

- repair or improve schedules before despatch whenever unforeseen events — creation/cancellation of tasks, early/late arrival of technicians, etc — make provisional assignments unfeasible or sub-optimal,
- support resource management by allowing the dynamic revision of business objectives and the implementation of ‘what-if’ scenario testing.

Dynamic Scheduler has been trialled successfully on different sites and has now been adopted as the standard allocation process of Work Manager. This paper describes the system from an engineering viewpoint where the emphasis is put on design issues rather than technicalities. The paper is structured as follows. Section 2 presents BT’s field-workforce scheduling problem (denoted WSP) and reviews possible approaches. Section 3 introduces the distributed model of Dynamic Scheduler. Section 4 presents the static computational model implemented to generate long-term schedules. Section 5 concludes on the benefits gained over RTA.

2. Workforce scheduling

Workforce scheduling is defined as the allocation of tasks to people over time which covers two distinct problems — static scheduling and dynamic scheduling. The former refers to the construction of schedules for scenarios where data is precise and complete. The latter refers to the construction of schedules for changing scenarios where the core data is imprecise and incomplete. Static scheduling involves the making of many complex decisions that have to result in feasible and good-quality schedules. As suggested in Fig 2, the number of decisions that can be made

and the number of schedules that can be generated for a static scenario get rapidly prohibitive so one has to resort to intelligent search techniques to find ‘acceptable’ schedules.

The added complexity of dynamic scheduling comes from the fact that the quality of a scheduling decision can never be guaranteed due to the uncertainty on data. Even if a decision is optimal for the data known at time t , it may be sub-optimal for the data known at time $t + 1$. For instance, choosing the closest technician to perform a task may turn out to be the wrong choice if the technician faces deteriorating traffic conditions on the journey.

2.1 Dynamic workforce scheduling

Field-workforce scheduling, being an open-world problem, clearly falls in the class of dynamic scheduling. This ‘dynamicity’ has different origins:

- the company and its customers who request, cancel or amend jobs unpredictably,
- the technicians whose availability is subject to last-minute changes and whose task completion times vary,
- the resource managers who may modify provisional schedules and review business objectives at any time,
- and finally, the environment itself which is inherently chaotic — weather, traffic conditions, etc.

Though the evolution of workforce scheduling scenarios follows specific patterns [2], there is no forecasting model realistic enough to allow for a static treatment of the dynamicity. Soft computing techniques [3] implement meta-knowledge concepts (e.g. possibility, probability, fuzziness) to model and reason about changes in a static

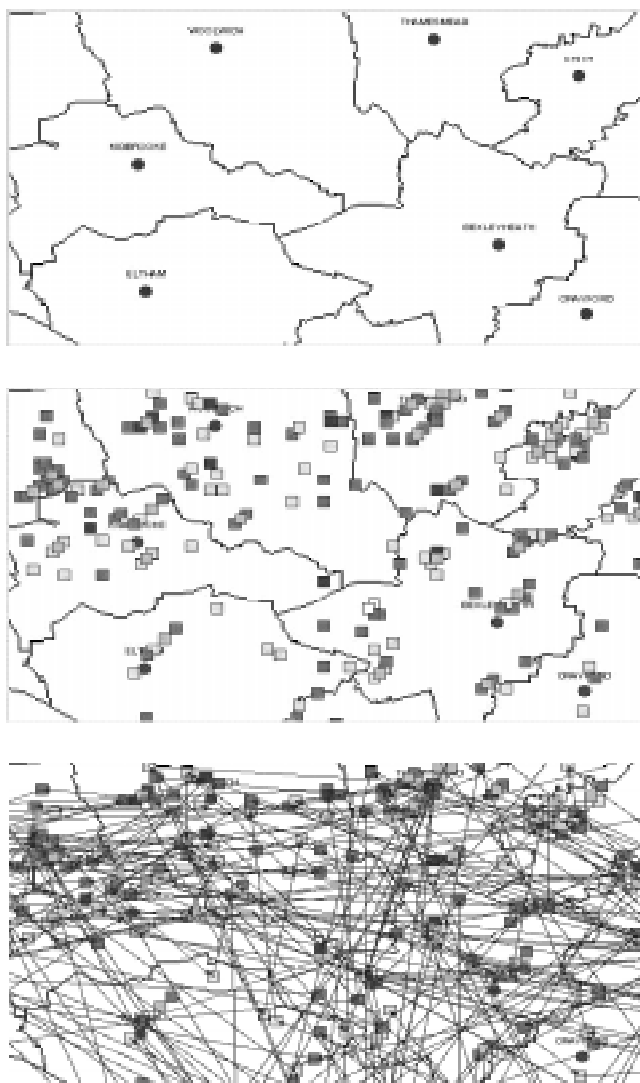


Fig 2 A WM domain (top picture), a distribution of tasks and technicians in this domain at a particular time (middle picture) and a possible schedule for this scenario (bottom picture).

way. However, this approach is not suited to the WSP because the complexity of its dynamics is not reducible to such knowledge engineering.

The common approach consists of addressing dynamic scheduling through rescheduling, i.e. by means of repeated executions of static scheduling operations. There are two possible strategies. On the one hand, predictive scheduling, i.e. the production of long-term schedules for the whole set of resources and tasks, enables one to achieve long-term optimisation. Unfortunately, it is a time-consuming operation which does not fulfil the need for responsiveness in highly dynamic environments. On the other hand, reactive scheduling, i.e. the fast production of short-term schedules for a limited number of resources and tasks, guarantees responsiveness at the expense of global optimality.

As for the WSP, which is highly dynamic but demands careful optimisation, the best strategy consists of coupling reactive and predictive scheduling. This can be done by employing a reactive component for on-line allocation and a background predictive component for global optimisation. In this case, the reactive component, being the ultimate decision-maker, is entitled to override any schedule released by the predictive component. In other words, any predictive work assignment may be ignored at despatch time. However, the intention is not that well-thought-out decisions be overridden for no reason, indeed the contribution of the predictive component to the whole scheduling exercise must be maximised.

The issue of coupling reactive and predictive scheduling in a satisfactory way has found no theoretical answer. Solutions are typically heuristic and implement opportunistic or mixed-initiative strategies [4], using distributed computational models based on blackboard architectures [5, 6]. Whatever model is retained, all the scheduling components must apply similar, if not identical, rules so as to achieve some form of compatibility and to maximise each component contribution. The next section is a detailed description of the static WSP that the predictive component is meant to address.

2.2 Static workforce scheduling

In its static version, the WSP is a complex variant of the well-known Vehicle Routing Problem (VRP) [7]. The VRP involves finding individual routes for a set of vehicles in order to visit different customer sites while satisfying various constraints and minimising a numeric function supposed to incorporate all user-defined quality criteria. The WSP differs from the VRP in the type of its constraints and criteria.

WSP constraints fall into the following categories.

- Geography:
 - each problem instance is tied to a domain controlled by an instance of Work Manager,
 - every technician has a working area within this domain and a predefined set of locations through which the route must pass (home, exchange, office, etc).
- Skills — every task demands a specific skill-set that only a subset of the technicians can provide.
- Access restrictions — access to customer premises may only be granted to certain individuals.

- Time restrictions:
 - technician availability — every technician is entitled to non-working periods (breaks, e.g. home time, lunch, personal absence, sick leave),
 - task operability:
 - some tasks are appointed, i.e. they cannot be executed outside the time windows agreed with the customer,
 - some tasks may be split over technician breaks, i.e. their execution is fragmented over a break or a few breaks,
 - duration variability — the duration of a task depends upon the assigned technician (their experience and skills), while the duration of a journey depends upon the origin and destination locations and the traffic conditions.
- Resource utilisation:
 - technicians only perform one activity at a time — executing a task, travelling, enjoying a break or waiting idle,
 - vehicles and stores are not shared, every individual being assigned a vehicle and equipped with the proper stores.
- Business rules enforce or relax existing resource utilisation constraints:
 - work in overtime may be decided to complete the last task of the day, or travel in own-time may be used when travelling to the first task or from the last task of the day,
 - task-splitting modalities depend on the type of work and the type of breaks.
- Task co-ordination (some tasks must be carefully synchronised for logical or technological reasons):
 - sequences, e.g. set-up before an operation (same individual), or progress review after an operation (different individuals),
 - line-ups — the simultaneous participation of different technicians to the same job at different places.
- Constraints inherited from the dynamic scheduling problem:
 - schedule horizon (the time window during which every task must be executed),
 - on-going activities with their estimated completion time,
 - pre-pinning of technicians, etc.

Constraints are meant to guarantee the feasibility of the schedules generated. Their semantic is simple — whenever an allocation is attempted, be it physical or temporal, all the constraints to which it is subject must be satisfied. Note that the scheduling of all the tasks is sometimes stated as a constraint itself. If so, the resulting constraint satisfaction problem is hard to solve and some elementary constraints may have to be relaxed to allow for ‘complete’ schedules — schedules where all the tasks are assigned — to be generated in acceptable time. As for the WSP, completeness is not modelled as a constraint but as a quality criterion — non-allocated tasks are accepted but their number should be minimised.

WSP quality criteria express the following concerns.

- Productivity:
 - maximise the number of tasks scheduled,
 - assign the most efficient technician to each task.
- Quality of service which is measured at the task level according to:
 - the customer type (residential, business, BT),
 - the work importance (provision, installation, maintenance, repair, etc),
 - the compliance with agreed due dates for non-appointed tasks — primary/secondary start/completion time.
- Minimisation of costs covering:
 - travel time between successive locations,
 - waiting time before opening of appointed-task windows and before breaks,
 - work in overtime.
- Satisfaction of technician preferences expressed on their working areas or their type of activity.
- Satisfaction of objectives inherited from the dynamic scheduling problem:
 - maximise schedule robustness in order to make schedule adjustments on despatch virtually cost-free,
 - maximise schedule flexibility in order to ease repair/improvement operations on despatch.

Each criterion reflects a viewpoint from which the quality of a schedule may be assessed. In this respect, a scheduling algorithm must generate schedules that maximise the quality along all the specified dimensions. This multi-criteria aggregation problem is undermined by multiple difficulties, namely:

- the vagueness surrounding certain criteria, e.g. quality of service, robustness, which makes their mathematical formalisation a delicate exercise,
- the coexistence of qualitative and quantitative criteria which are not easily amenable to a unified aggregation process,
- the latent conflict between criteria which demands adequate balancing, e.g. minimise makespan and costs while maximising productivity,
- the absence of clear-cut rules on the way aggregation should be conducted.

In contrast to the constraint satisfaction approach, there is no single computational model for multi-criteria aggregation that is universally accepted. Interactive scheduling, whereby the expert designs a solution with the support of a reasoning system that automates low-level operations, is one approach. The advantage is that the aggregation is performed by the user and does not need to be formalised. The drawback is that it cannot be implemented in time-constrained environments which demand fast solution design. For this reason, the aggregation process has to be automated. This is traditionally done by modelling all the criteria within a single cost function and by defining the multi-aggregation problem as the problem of minimising the function.

Regarding problem scale, BT WSP static instances all differ from one another according to the business domain, the operational environment, and the time of the day at which they are operating. This heterogeneity expresses itself in the number and degree of the constraints involved, in the weight of each criterion, and in the size of the core data. Regarding problem size, the number of technicians may range from a dozen to a few hundred, the number of tasks may range from a few dozen to a few thousand, while the schedule horizon may span over a few hours to a few days.

2.3 *Synthesis*

The following observations have been made regarding the way to address the dynamic WSP:

- rescheduling is the only alternative to tackle the dynamicity of the WSP,
- coupling predictive and reactive scheduling processes where the former is employed for on-line allocation and the latter for background optimisation is vital to achieving ‘informed responsiveness’,
- in this case, scheduling processes are subject to individual and collective requirements (logical consistency, short-response time, robust and flexible answers, etc).

These guidelines have been applied and a loosely coupled distributed model is advocated for Dynamic Scheduler [8], which is presented in section 3.

The static WSP has been described as a complex variant of the Vehicle Routing Problem featuring multiple vehicles, multiple depots, compatibility constraints (skill-matching, access restrictions), time constraints (time windows and task splitting), side-constraints (business rules, dynamic scheduling constraints), synchronisation constraints and multiple quality criteria. It has been suggested that a solution to this problem:

- must construct schedules that are consistent with all the constraints specified,
- is entitled to construct incomplete schedules,
- must address multi-criteria aggregation as an optimisation problem.

The predictive component of Dynamic Scheduler complies with these requirements and addresses the static WSP as a constrained VRP. The computational model is presented in section 4.

3. **Dynamic Scheduler — distributed model**

Work Manager hosts and integrates Dynamic Scheduler with the other work management processes and systems. Work Manager [9] provides the following features:

- a generic interface to other BT systems which provide it with work,
- a task jeopardy management system to ensure that managers are alerted to potential failure,
- out-of-hours (call-out) rota management system,
- task despatch and task progress/closure system, including time-recording facility,
- sophisticated feedback services to keep the system which originated a task, informed of its progress,
- management of work involving sequential or parallel dependencies between tasks,
- interface to inventory management systems,
- real-time and historic management information system (MIS) to provide measures of the performance of the application and of the workforce, etc.

The position of Work Manager with respect to BT’s job allocation and field despatch systems is illustrated in Fig 1.

Dynamic Scheduler features both predictive and reactive scheduling components which operate asynchronously. The predictive component is used for

global optimisation and is run in the background. It constructs provisional schedules periodically throughout the day. These schedules are fed to the reactive component, a new schedule overwriting the previous one. The periodic activation strategy has been adopted for its simplicity and because it is notably difficult to estimate the amount of computational resources needed to 'solve' unknown combinatorial problem instances.

The reactive component is in charge of the on-line allocation and is triggered by external events, e.g. work requests put in by on-line technicians, immediate allocation required for tasks on the verge of failing business targets. By default, it complies with the work assignments prescribed in the provisional schedule. However, it may implement its own assignments if recent changes have made the provisional schedule unfeasible or sub-optimal. The scope of this decision-making is limited to a subset of the resources and tasks, and to a short horizon so as to deliver acceptable response times (in the order of a second).

A schedule manager has been introduced to manage the interaction between these components and Work Manager. At regular time intervals, it extracts the appropriate data from Work Manager databases for the needs of the predictive component. It also stores and provides access to the provisional schedule for the needs of the reactive component. Note that this component does not carry out any scheduling activity and in particular does not control the predictive and reactive components. It simply acts as an interface between Dynamic Scheduler and Work Manager. Fig 3 shows the architecture of Dynamic Scheduler.

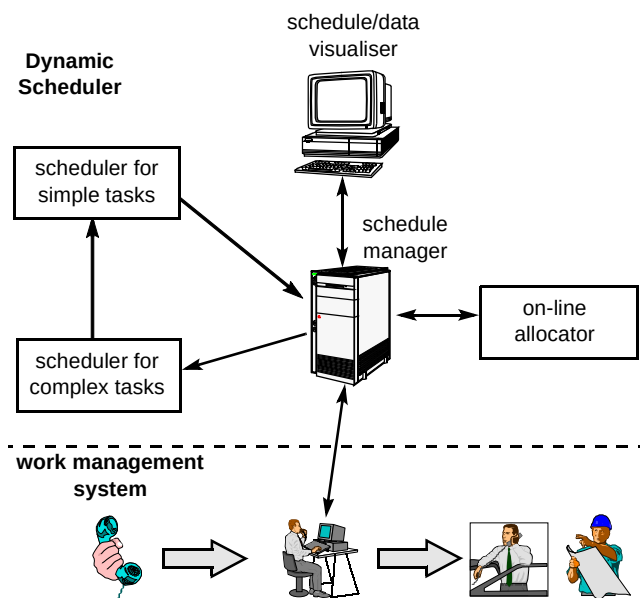


Fig 3 The Dynamic Scheduler architecture.

Finally, an interface allows for the visualisation of the provisional schedules and the information generated by the

scheduling components. Both temporal and spatial dimensions of the schedules are visible using Gantt charts, geographical map, pie-charts, bar-charts, and graphs (Fig 4). In addition, this tool allows for implementing what-if scenarios with a clone of the predictive component.

4. Dynamic Scheduler — predictive model

This section introduces the predictive scheduling component of Dynamic Scheduler which is dedicated to the resolution of static WSP instances. As advocated in section 2, the approach is to model the WSP as a constrained optimisation problem which consists of finding an assignment of the decision variables (representing the temporal and physical allocation of the tasks and technicians) that satisfies all the relevant constraints and optimises the specified cost function. This problem is a variant of the Vehicle Routeing Problem which is known to be highly combinatorial. Optimal solutions cannot be delivered in acceptable time and one has to resort to approximation techniques that can generate 'good' solutions in little time. The most successful approximation techniques implement Local Search principles.

Local Search is a search paradigm based on the notion of iterative improvement that has been introduced for solving pure optimisation problems. As such, Local Search techniques do not feature any modelling and computational capabilities for reasoning on constraints. Although these techniques have a proven track record on the academic VRP, the complexity of the constraints involved in the WSP make them inapplicable to this problem.

On the other hand, Constraint Satisfaction has emerged as the most powerful paradigm for representing and solving Constraint Satisfaction problems. Constraint Satisfaction techniques have been applied to many real-life problems as demonstrated by the success of constraint programming tools. However, in contrast with Local Search, Constraint Satisfaction techniques are not meant to solve optimisation problems and therefore cannot be applied to the WSP on their own.

Since Local Search and Constraint Satisfaction offer complementary techniques, it is tempting to combine them so as to tackle comprehensively all aspects of constrained optimisation problems. This is the approach that has been taken for the WSP and that has also been adopted in recent works on the VRP [10]. Our approach consists of:

- designing a solution representation that supports both Local Search and Constraint Satisfaction operations,

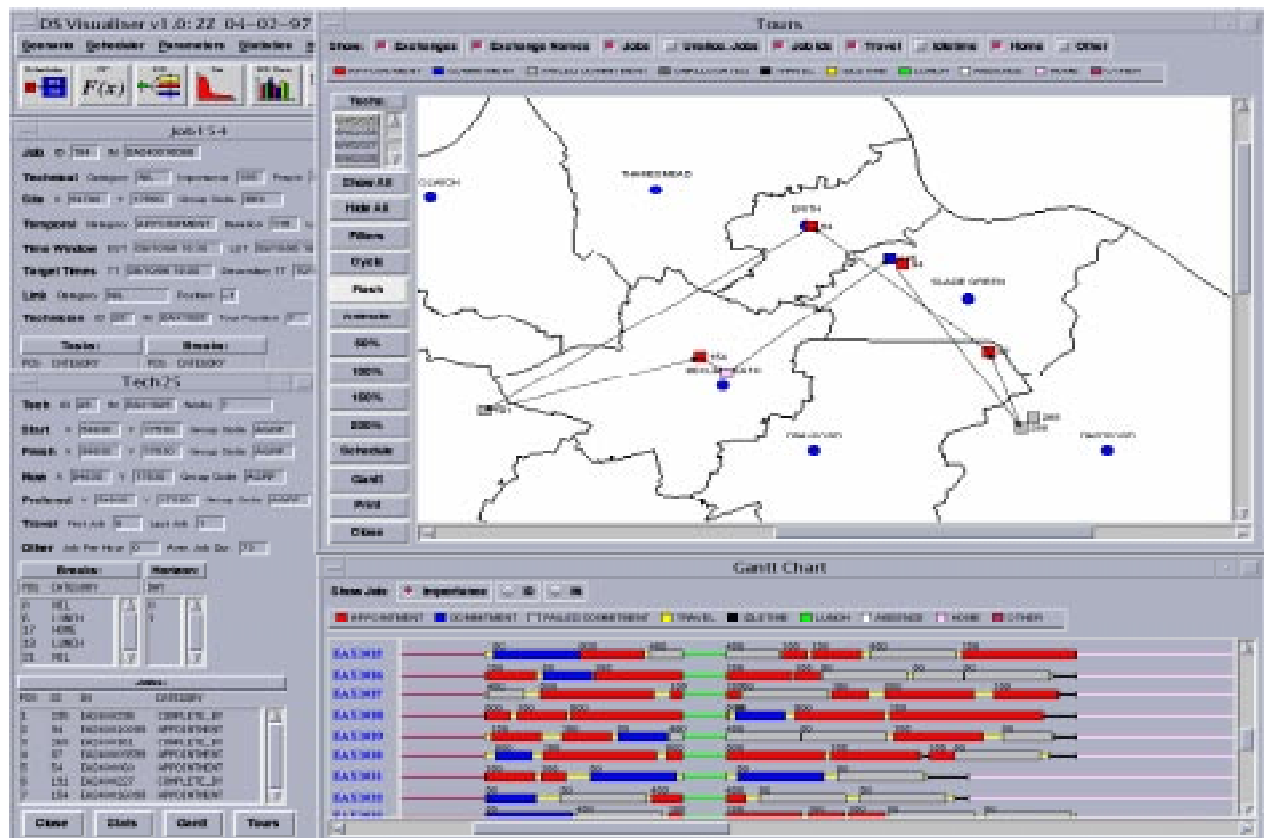


Fig 4 Example of schedule/data visualisation.

- decomposing the resolution into two stages where:
 - a systematic algorithm pre-schedules the complex tasks (long-duration tasks and tasks to synchronise),
 - a Simulated Annealing algorithm schedules the other tasks from this pre-schedule and releases the provisional schedule,
- each algorithm operates on the above-mentioned solution representation.
- the logical representation of solutions,
- the move-generation mechanism which specifies the type of changes allowed on the current solution and consequently the solutions that can be derived (the 'neighbours'),
- the acceptance criterion which specifies which solutions are acceptable in the neighbourhood of the current solution,
- the selection criterion which specifies the modalities of the exploration of the neighbourhood,
- the termination criterion which specifies the end of the search.

Firstly, the basic principles of Local Search are reviewed and the need for integrating Local Search with Constraint Satisfaction and for decomposing the problem is justified. Then the solution representation, the Local Search and Constraint Satisfaction operations it supports, and the search algorithms are introduced. A detailed description of this model can be found elsewhere [11].

4.1 Local Search

Local Search (LS) consists of starting from an initial solution, i.e. a complete assignment, and of repeatedly modifying it locally to obtain better solutions. The key attributes of an LS technique are:

As for the VRP, a solution is usually defined as a set of tours, each tour representing the sequence of tasks assigned to a technician over the schedule horizon. Several move-generation mechanisms have been devised that relocate or exchange tasks or sub-tours between technicians — *2-opt*, *relocate*, *exchange*, *cross* (see Fig 5).

Impact of constraints

Move operators were primarily designed for the unconstrained VRP whose objective is to minimise the total distance travelled. As a result, they present certain limits and side effects when applied to the WSP. For instance, *2-opt*, which reverses a sequence of tasks in a tour, does not

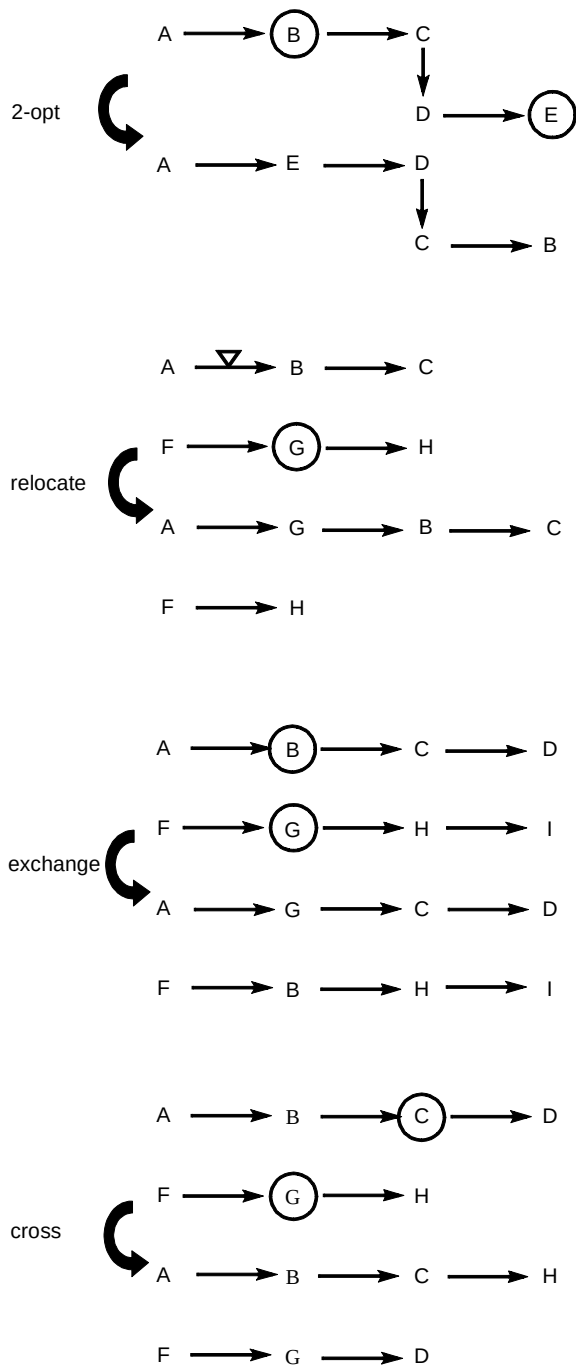


Fig 5 Move operators for the VRP.

preserve the orientation of tours. This is not desirable when task prioritisation is a concern. Furthermore, if the tasks affected by *2-opt* are appointed tasks with tight time-windows, the probability for the target solution to be inconsistent is higher. The same applies to the *exchange* operator that swaps two tasks from either the same or different tours, and, to a greater extent, to the *cross* operator which swaps the end portions of two tours.

In fact, an operator is more likely to generate inconsistent solutions if the number of tasks and technicians

it affects is high. Clearly, the wider its scope in terms of tasks and technicians affected, the wider the range and number of constraints it involves, the bigger the risk of causing inconsistency, and the higher the computational cost for checking consistency. Independently from the operators utilised, the efficiency and rapidity of an a Local Search will depend on how constrained the problem instance is as measured by the number and tightness of its constraints.

The issue raised here is that of finding a way that can limit the impact of constraints on the performances of LS techniques. The first solution is to cancel out the Constraint Satisfaction Problem by adopting a solution representation which implicitly satisfies the constraints. In the Travelling Salesman Problem for instance, solutions are usually represented as permutations of the cities to visit so the constraint that no city should be visited twice is automatically satisfied. As for the VRP, the tour-based representation aims to solve, or rather subsume, the resource utilisation constraints by sequencing the tasks. In the WSP, such sequencing is not necessarily consistent with the other constraints (time-windows, working hours, synchronisation constraints, etc).

A second solution consists of assimilating the constraints to criteria which means that the satisfaction of constraints is treated as an optimisation problem where constraint violation is permitted. This approach, however, has been discarded since WSP constraints are all meant to be satisfied. Furthermore, this would make the multi-criteria aggregation issue worse than it already is since constraint-related terms would have to be included in the cost function.

The chosen solution consists of coupling the LS process with a proper Constraint Satisfaction procedure where the former guides the exploration and the latter determines the consistency of each attempted solution. This is the best approach from an engineering viewpoint since it complies with the user specifications and it allows for the appropriate techniques to be used — constraints and criteria are differentiated and processed according to the semantics the user has assigned to them.

Of course, the viability of this approach depends on the complexity of the underlying constraint satisfaction problem. If the latter is intractable, there are only two options left. The first one consists of allowing the LS process for generating and exploring inconsistent solutions. As for the WSP, this is only acceptable if some repair capability is provided that enables the attainment of a consistent solution in the end. This approach raises a new set of issues regarding the nature of the repair operations and the modalities of their application.

It would appear to be simpler to prevent an LS process from visiting inconsistent solutions. In this case, the only option left is to decompose the problem by processing separately the constraints which are the source of intractability. Some of these constraints may not ultimately be satisfied but, at least, the other constraints that form the tractable part of the problem should be handled satisfactorily by the LS process. This is the option followed by decomposing the WSP into two sub-problems, hence allowing LS to be used at its best.

It should be added that constraints have also the potential to ‘disconnect’ the search space and make a local search ‘incomplete’. Indeed, the aim is to force the LS process to operate in the space of consistent solutions which is included in the whole space of solutions. However, the shape of this subspace is not known. For instance, it may contain solutions for which there is no succession of moves that allows reaching one from the other.

If so, the exploration will be limited to only the parts of the space that are attainable from the initial solution and good solutions may be missed. Although LS is not meant to be exhaustive, it is desirable that all the solutions are accessible from one another so as to limit the impact of the initial solution on the output of the search. There again, the extent of the issue will depend on how constrained the instance is.

Local minima

The acceptance and selection criteria differentiate LS techniques from one another. Typically, the acceptance criterion requires new solutions to improve on the cost of the current solution.

LS is based on the belief that the search should progress towards optimal solutions if it follows the gradient defined by the cost function. However, it should be borne in mind that the set of solutions to the problem is not a structured space but a mere collection of points. It is in fact the move operator(s) which structures the search space by ‘assigning’ a neighbourhood to each solution. The profile of the cost function in this structured space forms what is improperly called the landscape of the search space.

Depending on the cost function and on the move operator, the landscape may be more or less undulating and may contain more or less local minima, i.e. solutions which have the minimum cost in their neighbourhood but which are not optima. Because LS is essentially a gradient descent, it is attracted by local minima. This implies that much CPU time may be wasted if the search process is misled by such points.

The most natural way of reducing the occurrence of local minima is to design a cost function that makes the landscape as smooth as possible. Although the cost elements representing the quality criteria and their prioritisation are dictated by the user, there is still room for the designer to conceive a sound mathematical framework to formalise the cost function. Yet, this is not sufficient and a back-up solution to the occurrence of local minima is to use techniques that can escape from local minima. Simulated Annealing is one of those.

4.2 Coupling Local Search and Constraint Satisfaction

Coupling Local Search and Constraint Satisfaction means that two equivalent representations of the solution/schedule have to be managed during the search so as to allow for move generations on the one hand and constraint solving on the other. Each representation is described along with the operation it supports.

Local Search schedule representation

The tour-based representation is adopted, whereby each task is assigned a technician and a position. To enable the representation of incomplete schedules, a virtual technician was also introduced to hold all the tasks left non-allocated.

Move operators

The operator *relocate* is used which relocates a task either in the same tour or in a different tour. *Relocate* ensures a weak form of ‘completeness’ for the LS process. By authorising the relocation of tasks in or from the virtual tour, it is theoretically possible to reach every consistent schedule from any other consistent schedule (any schedule may be deconstructed by de-allocating all the tasks, hence generating the empty schedule and any schedule may be generated incrementally from the empty schedule). Above all, the restricted scope of *relocate* is better suited than the scope of *2-opt*, *exchange*, *cross* or *k-opt* — these operators lose much efficiency when confronted with the constraints of the WSP.

Cost function

The cost function is defined as a weighted sum of measures, each encoding a global criterion — quality of service, travel time, work in overtime, individual preferences, etc. Each measure is itself a weighted sum, e.g. the QoS is the weighted sum of the QoS measured for each task in the schedule. An individual measure, say the QoS for a task, is obtained by evaluating a normalised function that accounts for the attributes of the entity, e.g. customer type, work category or target dates for a task, overtime budget or skill preferences for a technician.

The weighted sum operation has been adopted as the numeric aggregation process for its intelligibility, for the cost normalisation it achieves and for the parameterisation it enables. The user may indeed set the values of the global and individual weights so as to achieve the required balance at every level. Normalisation is an essential feature that helps to deliver a 'harmonious' coupling between the cost function and the operators.

An important property is that the cost of a schedule can only be measured once the start times of the tasks are computed by the constraint satisfaction mechanisms. In other words, consistency checking must always be performed before cost evaluation and no solution can ever be discarded due to its cost before it is actually proved feasible.

Constraint-based schedule representation

Constructing a schedule implies the making of two decisions for each task — the choice of a technician, then the choice of a position in the tour of the technician. Each choice must be validated by checking the appropriate constraints. The constraints are thus separated into two classes, namely the compatibility constraints which restrict the matching of tasks and technicians, and the temporal constraints which restrict the positioning of tasks.

Each class gives rise to a Constraint Satisfaction Problem (CSP) which is formally defined by a set of variables, a finite domain for each variable, and a set of constraints that restrict the values the variables may take either individually or collectively. Solving a CSP means finding a solution — an assignment of the variables that satisfy all the constraints — or proving that no solution exists, i.e. proving that the problem is not consistent.

Whenever LS generates a schedule, the 'compatibility CSP' and then the 'temporal CSP' must be solved. Each problem is defined in turn and their representation and resolution discussed.

Solving compatibility constraints by preprocessing

The compatibility CSP is defined by associating a variable to each task that represents the technician to whom the task is assigned in the schedule. The initial domain of the variable is the whole set of technicians and includes the virtual technician. Most compatibility constraints restrict the set of technicians independently of the scheduling of the task. These are the skill-matching constraints, the access restrictions, and the time restrictions on the task and its candidate technicians, namely schedule horizon, time-windows if the task is appointed, working hours of the technicians, travel time between breaks, business rules on the usage of travel in own time, and work in overtime and on the modalities of task splitting.

All these constraints are solved before the search starts by removing the incompatible technicians from each domain. Skill matching and time restrictions are easily solved, but time restrictions must determine the impact of technician breaks on each activity (each allocation task-technician). A break has three possible effects on an activity — either it prevents its execution, or it enforces the splitting of the execution, or it has no effect at all. Consequently, the modalities of the execution vary in time; the activity may not be feasible at certain times, it may be carried out as normal at others, or it may have to be split over a break or a series of breaks.

It should be noted that all the breaks have fixed start and end times and do not have to be scheduled as tasks do. This makes the problem much simpler than general pre-emptive scheduling problems wherein tasks may be split over other tasks. The preprocessing of the time restrictions on an activity consists of intersecting the task and technician windows — an activity cannot be executed during a break — and of computing the impact of each break on the resulting time windows. This consists of checking whether the activity may be started before the break and, if it has to be split over the break, whether the modalities of the split comply with the business rules. If ultimately no start time is feasible over the schedule horizon, the activity is considered unfeasible and the technician is removed from the domain of the task.

Preprocessing records the result of each constraint check by a physical deletion in a domain. The interest is that time-consuming procedural calls will be avoided if the constraints have to be checked many times during the course of a search which is the case in LS where the same task may be relocated many times.

Overall, preprocessing is profitable if its run time does not exceed the CPU time it saves subsequently during the search. This is largely verified in this case — the preprocessing procedure has a linear worst-case complexity $O(e.t.d.w)$, where e is the number of technicians of the problem, t is the number of tasks, d is the number of days covered by the schedule horizon, and w is the largest of the maximum number of time-windows per day for a task and the maximum number of breaks per day for a technician.

The remaining compatibility constraints are binary and reflect follow-up/set-up requirements whereby two tasks must be assigned to the same individual but the individual is unspecified. Such constraints need to be checked whenever one of the tasks is relocated and these checks take constant time. To sum up, the majority of compatibility constraints are preprocessed which not only enables the LS process to avoid slowing down, but also helps in reducing the search space beforehand.

Solving temporal constraints with arc-B-consistency

The temporal CSP only makes sense if the LS schedule is consistent with the compatibility constraints. This CSP associates four temporal variables to each allocated task. The non-allocated tasks are not assigned any variable which implies that synchronisation constraints on these tasks are ignored. For each task i , the following variables are introduced:

- two variables s_i and e_i representing respectively the start and end times of i ,
- two variables a_i and d_i representing respectively the arrival time on the site of i and the departure time from the site of i of the technician assigned to i .

These variables are assigned the same initial domain which is the discrete time interval defined by the schedule horizon. The grain of this interval is the one-minute grain used in Work Manager processes. The preprocessing stage mentioned above reduces each domain by removing the dates which are inconsistent with the time constraints. This reduction results in a fragmentation of the initial interval into disjoint intervals.

The other temporal constraints are binary. Firstly, a constraint is stated between the start and end time of an activity so as to reflect the impact of breaks. Indeed, the overall duration of an activity — the time elapsed between its start and end times — is increased if the activity is split and the extent of the increase depends upon the number of breaks overlapped, their duration and the ineffective time (travel time and waiting time) induced. This relationship is computed during preprocessing and is represented by a sequential timetable [12, 13]. The timetable records the evolution of the activity's duration and other cost-related elements along with its start and end times.

The other constraints are as follows. For each activity i , the following constraints are introduced:

- a constraint $a_i \leq s_i$ to express that the technician must be on-site to start the activity,
- a constraint $e_i \leq d_i$ to express that the technician cannot leave the site of the activity before completion.

For every pair of activities i, j scheduled successively in a tour, a constraint between d_i and a_j is stated to express that the technician has to travel between the two sites. Note that the travel time (the difference $a_j - d_i$) is variable, as the journey may be disrupted by breaks. Precisely, the travel time depends upon the departure date or the arrival date specified. In contrast with activity duration, this relationship

is not pre-processed as it would be too prohibitive to consider every ordered pair of tasks for every possible technician. Finally, synchronisation constraints are stated which take the form of equalities on the start times of two activities, $s_i = s_j$, or inequalities between their start and end times, $e_i \leq s_j$.

To sum up, the temporal CSP is a binary CSP with fragmented integer domains which is represented using intervals, timetables and procedural definitions. The fundamental feature of this problem is its tractability. Indeed, all the constraints are binary min-closed and max-closed constraints [14] and one can easily show that a min-max-closed binary constraint network may be solved using arc-B-consistency enforcement [15].

Arc-B-consistency is an approximation of arc-consistency, the enforcement of which consists of propagating the constraints on domain bounds only. Applied to a min-max-closed binary CSP, it will either prove that the problem has no solution or it will reduce the domains of the variables such that the minima of the domains are guaranteed to be a solution to the problem as well as the maxima of the domains. As for the temporal CSP, the minima of the start-time variables will provide the earliest feasible start times of the tasks in the schedule while the maxima of the end-time variables will provide the latest feasible completion time of the tasks. Note that the feasibility of intermediate values in the domains (arrival/start/completion/departure time) is not guaranteed. Arc-consistency would have to be used for this purpose.

Decomposing the resolution

The worst-case time complexity of arc-B-consistency enforcement is $O(h.m)$ where h is the size of the schedule horizon and m the number of binary constraints while the best-case complexity is $\Omega(m)$. In the temporal CSP, each variable is involved in a maximum of four binary constraints. Thus, the worst-case and the best-case of arc-B-consistency become $O(h.t)$ and $\Omega(t)$ where t is the number of tasks allocated. Yet, given the size of the schedule horizon (recall that a one-minute grain is used to represent several days), the worst-case must be avoided.

The worst-case occurs whenever the schedule contains a temporal cycle which makes it inconsistent, e.g. task A precedes task B in a tour while task C , which is allocated to another technician, must be completed before A and after B . Cycles can be detected by running a depth-first search procedure on a directed constraint graph of the problem. This graph is derived from the constraint graph of the CSP by directing the edges representing inequality constraints and merging the nodes/variables involved in equality-like constraints.

The interest of depth-first search is that its worst-case complexity does not depend on the factor h ; it is simply linear in the number of tasks allocated in the schedule. However, it must be applied before arc-B-consistency enforcement for each move attempted. This will systematically slow down the neighbourhood exploration and may not be affordable since LS must perform the largest possible number of iterations during a run.

Another strategy consists of decomposing the temporal CSP resolution by pre-scheduling the tasks responsible for the occurrence of cycles and by scheduling the other tasks with LS. Cycles are caused by the tasks involved in synchronisation constraints like line-ups or sequences. Once these tasks are pre-scheduled consistently, they can be made 'transparent' to LS by preventing their relocation during moves. In this way, no cycle can ever be created and arc-B-consistency enforcement will never attain its worst case.

One may also fix the start and end times of the pre-scheduled tasks to speed up arc-B-consistency during LS. Thus, these tasks will not be viewed as variables and no propagation will be carried out along the synchronisation constraints — the propagation will be restricted to the tour into which the relocated task has been inserted. Another option is to allow for the temporal shifting of the pre-scheduled tasks in their tour; their time windows may be enforced or relaxed during arc-B-consistency so as to accommodate the allocation of other tasks. This option provides additional flexibility by enlarging the search space of the problem addressed by LS but it also makes consistency enforcement slower since synchronisation constraints must be propagated and, potentially, every tour may be affected.

The decomposition strategy has been adopted instead of the systematic cycle-detection for it preserves the applicability of LS by making consistency enforcement as fast as possible. For the same reason, the option consisting of fixing both the time and the technician of the pre-scheduled tasks has been preferred.

Constructive search for pre-scheduling

The pre-scheduling is performed by a systematic search which generates the initial solution for the LS process. It uses the solution representation, the move operator and the arc-B-consistency procedure mentioned above to allocate tasks and to check for schedule consistency. In contrast with LS, this pre-scheduling implements a backtracking search which theoretically guarantees to exhaust the search space, although this result has no practical value given the complexity of the problem.

Two sets of heuristics are implemented to guide the search. The first set consists of variable-ordering heuristics which dictate the choice of the next task to assign. The second set consists of value-ordering heuristics which dictate the choice of the technician and the position for the task to allocate. Tasks are ordered according to their importance, their urgency and how constrained they are as measured by the number of possible technicians, their duration, target times, etc. Technicians and positions are ordered according to the level of QoS and cost they induce.

Due to the backtracking strategy, some tasks may be left non-allocated. This essentially depends on the number of tasks to synchronise which can be close to zero in some domains. However, because long schedule horizons are in operation, it is usually easy to reach a complete pre-schedule. In any case, it is still possible to schedule the non-allocated tasks in the second stage by enhancing the LS constraint satisfaction engine with the appropriate cycle-detection procedure.

Simulated Annealing for optimisation

Simulated Annealing has been adopted as the LS technique to implement the second-stage scheduling. This choice has been motivated by the success of extensive experiments comparing other approximation techniques including Hill-climbing, Tabu Search and Genetic Algorithms on a simplified version of the WSP [16]. Designed by analogy with the process of annealing in statistical mechanics¹ [17], Simulated Annealing may be described as a refinement of Monte-Carlo search which proceeds as follows.

A neighbour S' of the current solution S is selected, either in a random or systematic way, and the cost difference $\partial = \text{cost}(S) - \text{cost}(S')$ is measured. If it is positive then the move is logically accepted. Otherwise it is accepted with a probability which decreases when ∂ increases. Thus, the greater the cost difference, the more likely a 'negative' move is to be rejected. Simulated Annealing ties the probability of accepting negative moves to a parameter T , called temperature, through the formula:

$$P = e^{(-\partial/T)}$$

The temperature decreases with time and so does the acceptance probability.

It is common to allow Simulated Annealing to reach equilibrium before reducing the temperature and this is

¹ The annealing process consists of heating a solid at high temperature and then of cooling it slowly to obtain a regular crystal structure.

measured as completing a specified number of iterations (moves). The parameters of Simulated Annealing are precisely:

- the start temperature,
- the number of iterations per temperature,
- the temperature reduction factor.

In an ideal setting of the parameters, the acceptance probability should be close to 1 at the highest temperature and close to 0 at the lowest temperature. This way, Simulated Annealing starts acting as a pure random search and ends as a pure gradient descent, the idea being that vast parts of the search space should be explored in the first stages so as to identify the most promising areas, the intensive exploration of which will be conducted in the later stages.

The probabilistic acceptance criterion of Simulated Annealing is combined with a first-accept selection criterion which consists of selecting a task and a technician randomly. If the task is non-appointed, the position is also selected randomly and the move is attempted. If the task is appointed, every position in the tour of the technician is attempted in order to minimise the risk of inconsistency caused by the time-windows. An exhaustive neighbourhood exploration considering all the technicians for the task selected would be far too prohibitive.

Finally, it should be pointed out that Simulated Annealing attempts to reuse the provisional schedule it generated in the previous run so that schedule continuity and stability can be maintained between successive runs. The previous schedule is simply replicated and combined with the pre-schedule before the search starts. This saves much CPU time since the search does not start from the empty schedule and, if few changes have affected the problem, this starting point should already be of good quality.

5. Conclusions

Dynamic Scheduler has been tested in different Work Manager domains. These field trials have confirmed the superiority of the approach over the purely reactive scheduling strategy based on RTA in the following areas:

- quality of service and productivity have been improved at constant costs,
- the levels of manual intervention have been reduced,
- resource management has been supported more effectively.

Furthermore, Dynamic Scheduler has contributed to increasing the scope and scalability of Work Manager.

This paper has highlighted the modelling and computational benefits gained by combining Local Search

and Constraint Satisfaction to address real-life constrained optimisation problems such as workforce scheduling. The paper has also shown that there is a long way from a theoretical model to its application — each decision on the design of Dynamic Scheduler (whether on the preprocessing of complex constraint representations, the formal framework of the cost function, the variable and constraint ordering heuristics, the consistency maintenance techniques, the problem decomposition strategy, or the search algorithms) has been crucial to the success of the system and has required a high level of operational design intelligence.

This stresses the importance of flexible constraint optimisation software that not only provides the user with generic problem modelling and solving capabilities but also allows the customisation and fine-tuning of the core resolution mechanisms.

Acknowledgments

This work is the result of a collaboration between various BT teams. The authors particularly wish to thank Ian Alleston, Andy Noble, John O'Donoghue and Christos Voudouris for their contributions.

References

- 1 Laithwaite R: 'Work allocation challenges and solutions in a large-scale work management environment', *BT Technol J*, **13**, No 1, pp 46—54 (1995).
- 2 Simpson J, Noble A, Egan B F, Morton D, Richards T and Burstin M: 'Experience in applying OR techniques to the solution of practical resource management problems', *BT Technol J*, **13**, No 1, pp 38—45 (1995).
- 3 Azvine B: and Wobcke W: 'Human-centred intelligent systems and soft computing' *BT Technol J*, **16**, No 3, pp 125—133 (July 1998).
- 4 Smith S F: 'OPIS: a methodology and architecture for reactive scheduling', in Zweben M and Fox M S (Eds): 'Intelligent scheduling', pp 29—66, Morgan Kaufmann Publishers, San Francisco, California (1994).
- 5 Collinot A, Le Pape C and Pinoteau G: 'SONIA: a knowledge-based scheduling system', *Artificial Intelligence on Engineering*, **3**, No 2, pp 86—94 (1988).
- 6 Hildum D W, Sadeh N M, Laliberty T J, McA'Nulty J, Smith S F and Kjenstad D: 'A blackboard-based architecture for supporting mixed-initiative management of integrated problem-solving tasks'. Proceedings of the 9th Conference on Innovative Applications of AI (IAAI-97) (1997).
- 7 Laporte G and Osman I H: 'Routing problems: a bibliography', *Annals of Operations Research*, **61**, pp 227—262 (1995).
- 8 Lesaint D, Voudouris C, Azarmi N and Laithwaite R: 'Dynamic workforce management', *IEE Colloquium on AI for Network Management Systems*, London (April 1997).
- 9 Garwood G J: 'Work Manager', *British Telecommunications Eng J*, **10**, pp 204—210 (1991).

- 10 De Backer B, Furnon V, Kilby P, Prosser P and Shaw P: 'Solving vehicle routing problems using constraint programming and meta-heuristics', *Journal of Heuristics*, 1, No 16 (1997).
- 11 Lesaint D: 'Workforce scheduling with Local Search and Constraint Programming: a case study', presented at INFORMS National Meeting, Montreal (April 1998).
- 12 Le Pape C: 'A universal constraint-based representation of time-tables: benefits, costs and ... benefits', *Proceedings AAAI-SIGMAN workshop on knowledge-based production planning, scheduling and control, 13th International Joint Conference on Artificial Intelligence (IJCAI-93)*, Chambéry, France (1993).
- 13 Le Pape C: 'Implementation of resource constraints in ILOG SCHEDULE: a library for the development of constraint-based scheduling systems', *Intelligent Systems Engineering*, 2, No 3 pp 55—66 (1994).
- 14 Jeavons P G and Cooper M C: 'Tractable constraints on ordered domains', *Artificial Intelligence*, 79 pp 327—339 (1995).
- 15 Lhomme O: 'Consistency techniques for numeric CSPs', *Proceedings of the 13th International Joint Conference on Artificial Intelligence (IJCAI-93)*, pp 232—238 (1993).
- 16 Brind C, Muller C and Prosser P: 'Stochastic techniques for resource management', *BT Technol J*, 13, No 1, pp 55—63 (1995).
- 17 Kirkpatrick S, Gelatt C D and Vecchi M P: 'Optimisation by simulated annealing', *Science*, 220, pp 671—680 (1983).



David Lesaint holds an MSc and a PhD in Artificial Intelligence from l'Ecole Nationale Supérieure de l'Aéronautique et de l'Espace (France). He has developed in his thesis a dual theory of Constraint Satisfaction problems for computing compact solution sets.

He joined the Intelligent Systems Research group in January 1995 and since then has worked on the application of AI to various problems including Field Workforce Scheduling and Feature Interaction.

He has designed and developed the planning algorithms of Dynamic Scheduler which was awarded an IT finalist medal by the British Computer Society in 1997.



Nader Azarmi manages the Intelligent Systems Research group at BT Laboratories. He joined BT in December 1989, working on the application of AI techniques to telecommunications network management. He went on to set up a research and development programme in the application of AI planning and scheduling technology to BT's resource management problems. He is currently managing an international AI and Machine Intelligence research programme. He received a BSc (Hons) in Computer Science and Mathematics from the University of Manchester, an MSc in Artificial Intelligence and a MPhil in Computing Science from Essex university. He is currently completing a PhD in Computer Science at Imperial College, London University.



Bob Laithwaite joined BT in January 1987 having previously undertaken research into automated lay-out of VLSI circuits using AI techniques and also spent 10 years with the MOD as a systems/electronics engineer.

Initial work at BT concentrated on research into work allocation and resource scheduling, from which followed system design responsibilities for an out-of-hours call-out system and a semi-automated network work allocation system. This led on to wider design responsibilities for the Work Manager system with particular interest in system performance, generic work interfaces and the work allocation algorithms. Recent work as chief designer has focused on extending the scope of this system (which supports over 20 000 users) by delivering an improved, and more flexible allocation capability, the 'Dynamic Scheduler', to suit BT's changing needs.



Paul Walker joined BT in 1969 and spent 13 years in the local network and exchange repair environment. From 1984, he managed an integrated Repair Service Control and Customer Reception Centre which became the lead site for the first computerised administration system. He was a lead player in the district rationalisation programme and managed the integration and reduction of RSCs and the implementation of the fore runner to CSS repair handling. He introduced Customer Service Officers into the Manchester operation and became Manager of Manchester's centralised fault reception centre in 1988. He joined the Work Management Programme shortly after the programme's inception in 1990 and began creating Work Manager. He won the BT Gold Medal in 1997 for his significant contribution to BT with Work Manager, and a British Computer Society IT finalist medal for his contribution to the WM Dynamic Scheduling system.